

Camera-LiDAR Based Autonomous Navigation in BARN using ROS2

Muhammad Umair
Student ID: 25280013

Ahmed Adil
Student ID: 25280033

Muhammad Saad
Student ID: 25280089

Abstract—This paper presents a ROS 2-based autonomous navigation system for the Benchmark Autonomous Robot Navigation (BARN) environment, combining vision and LiDAR sensing for robust navigation in cluttered spaces. Our vision-based masking pipeline segments obstacles and extracts traversable floor regions from RGB images, using a centroid-based proportional controller to guide the robot. Simultaneously, LiDAR data is processed via a Vector Field Histogram (VFH)-inspired approach. This method constructs an inverse-distance weighted polar histogram to identify contiguous free-space regions, selecting the midpoint of the widest feasible valley as the optimal path. The final steering command is a weighted fusion of these dual perception streams, ensuring robustness under varying environmental conditions. To overcome local minima, we incorporate SLAM for global mapping and an exploration-driven scoring function for effective recovery. The system achieves an average score of 0.324 on the BARN benchmark and successfully navigates 41 out of 50 worlds (250-299) of the test worlds, demonstrating significant effectiveness in handling narrow passages and occlusions.

Index Terms—Autonomous Navigation, LiDAR, Camera, Proportional Controller, BARN Challenge

I. INTRODUCTION

While navigation in open and structured environments has advanced significantly, navigating through dense obstacle spaces remains challenging for mobile robots. Traditional methods relied mainly on 2D LiDAR, but this is often insufficient in complex environments. As a result, modern approaches use multi-modal systems that combine vision and LiDAR [8]. LiDAR provides accurate depth information, while vision adds semantic understanding, allowing robots to better detect obstacles and identify safe paths.

II. LITERATURE REVIEW

Autonomous mobile robot navigation has traditionally been addressed through a combination of *global* and *local* planning algorithms, which together form the foundation of classical collision avoidance strategies. Global planners assume access to a complete map of the environment and compute an optimal path from start to goal using algorithms such as A*, Dijkstra, and sampling-based methods like RRT and PRM. While these methods provide optimal or near-optimal solutions, they are less suitable for highly constrained or unknown environments—such as those encountered in the Benchmark Autonomous Robot Navigation

(BARN) challenge [7], [8]—where a complete map is not available in advance.

In contrast, *local planning algorithms* operate reactively using real-time sensor data, making them more suitable for dynamic and partially observable environments. One of the most widely used approaches is the Dynamic Window Approach (DWA), which performs collision avoidance by searching directly in the robot's velocity space while respecting kinematic and dynamic constraints. It evaluates admissible velocity commands based on progress toward the goal, obstacle clearance, and forward velocity, enabling safe real-time navigation [1].

Another influential method is the Vector Field Histogram (VFH), which constructs a local occupancy grid from sensor data and converts it into a polar histogram representing obstacle density in different directions. The robot then selects a collision-free steering direction by identifying low-density sectors while considering its kinematic constraints. VFH enables fast and smooth navigation in real-time environments, though it may struggle in very narrow passages or highly cluttered spaces due to histogram thresholding limitations [2].

Another commonly used technique is the Artificial Potential Field (APF) method, where the robot is guided by a virtual force field composed of attractive forces from the goal and repulsive forces from obstacles. This method is computationally efficient and suitable for real-time applications, but it suffers from well-known issues such as local minima, where the robot may become trapped in non-goal configurations [3].

The Velocity Obstacle (VO) approach extends collision avoidance into the velocity domain by identifying unsafe velocities that would result in future collisions. By selecting velocities outside these regions, the robot can safely navigate dynamic environments while accounting for both its own motion and that of obstacles [4].

Another class of lightweight reactive methods is the family of Bug algorithms, which rely on minimal sensing and simple behavioral rules. These algorithms guide the robot toward the goal and switch to boundary-following behavior upon encountering obstacles. Variants such as Bug1 and Bug2 guarantee goal reachability under certain assumptions but often

produce suboptimal paths. More advanced variants like DistBug and TangentBug incorporate distance measurements and range sensing to improve efficiency. However, many Bug algorithms assume accurate localization, which is difficult to achieve in real-world environments with sensor noise [5].

Beyond purely reactive methods, graph-based approaches such as centroid-based cell decomposition provide a structured representation of the environment by partitioning it into smaller regions and constructing a connectivity graph. Recent work combines this with bio-inspired optimization techniques such as Ant Colony Optimization (ACO) to refine paths and improve traversal efficiency. This hybrid strategy enables near-optimal path generation while maintaining computational tractability in complex environments [6].

Despite their efficiency, classical methods often struggle in highly cluttered or narrow environments due to issues such as local minima, oscillatory behavior, and limited foresight. To evaluate such edge cases, the Benchmark Autonomous Robot Navigation (BARN) dataset was introduced [7], and subsequent evaluations in the 3rd BARN Challenge at ICRA 2024 further highlighted the limitations of traditional planners in highly constrained environments [8].

Modern navigation frameworks increasingly rely on machine learning techniques such as imitation learning and reinforcement learning. For instance, the LiCS framework utilizes behavior cloning to learn navigation policies from expert demonstrations, improving robustness in cluttered environments [9]. While these approaches offer strong generalization capabilities, they often require large datasets and significant computational resources.

In summary, classical navigation algorithms provide a strong foundation for real-time collision avoidance, particularly in resource-constrained systems. However, their limitations in highly constrained environments—clearly demonstrated in BARN challenge evaluations—motivate the development of hybrid approaches [8]. In this work, we adopt a hybrid navigation strategy that combines VFH-based gap detection for reactive LiDAR steering, camera-based floor centroid tracking for path following, and a SLAM-guided occupancy scoring layer to break out of local minima. The proposed fusion mechanism dynamically weights sensor contributions based on proximity, aiming to balance moment-to-moment responsiveness with the higher-level spatial awareness needed to handle dead ends and revisited regions in cluttered environments.

III. PROBLEM SETUP

The Benchmark Autonomous Robot Navigation (BARN) ecosystem serves as the definitive proving ground for evaluating these navigation stacks [7], [8]. The problem setup is strictly defined by standardized hardware, randomized map generation, and a rigid scoring metric.

A. TurtleBot 3 Waffle Pi

All algorithms are evaluated on the TurtleBot 3 Waffle Pi platform in Gazebo simulation using ROS 2. The platform is equipped with a 360° 2D LiDAR and an RGB camera. [10]

B. Environment Configuration

The simulation evaluates algorithms across 300 diverse environments to analyze their ability to generalize. [8]. The scenarios vary from dense pillar fields to narrow, maze-like corridors, intentionally introducing local minima and challenging, out-of-distribution configurations [8]. Strict rules prohibit any prior mapping, requiring systems to rely entirely on real-time perception and reactive path planning [7].

C. Scoring Metric

The challenge uses a standardized performance metric that accounts for navigation success, traversal time, and environment difficulty. The score s_i for navigating each environment i is computed as:

$$s_i = \mathbf{1}_i^{\text{success}} \times \frac{\text{OT}_i}{\text{clip}(\text{AT}_i, 2\text{OT}_i, 8\text{OT}_i)} \quad (1)$$

where the indicator function $\mathbf{1}_{\text{success}}$ evaluates to 1 if the robot reaches the navigation goal without any collisions, and 0 otherwise. AT denotes the actual traversal time, while OT denotes the optimal traversal time—an indicator of environment difficulty measured as the shortest traversal time assuming the robot always travels at its maximum speed (2 m/s):

$$\text{OT}_i = \frac{\text{Path Length}_i}{\text{Maximal Speed}} \quad (2)$$

Clipping the actual traversal time between 2OT_i and 8OT_i penalizes slow traversals while setting a floor to avoid artificially high scores on trivially short paths.

IV. PROPOSED METHODOLOGY

A. Perception

The system utilizes a dual-modality perception architecture, decoupling macro-navigation (path following) from micro-navigation (obstacle avoidance) to optimize computational efficiency and system robustness.

1) *Camera based Masking*: The robot uses a camera to get RGB images and converts them to HSV color space. This transformation separates color information (hue) from brightness (value), making it easier to detect objects reliably even in shadows or uneven lighting conditions.

Once the converted images a mask is applied to detect objects and floor. This mask is then inverted so that the obstacle are black pixels and the floor is white pixels. The masking operation composed of union of three different mask is shown in the image and formula given below:

$$M_{\text{floor}} = \neg(M_{\text{red}} \cup M_{\text{blue}} \cup M_{\text{shadow}}) \quad (3)$$

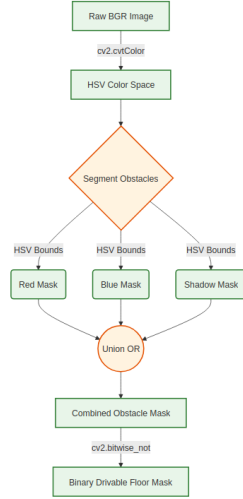


Fig. 1. Masking

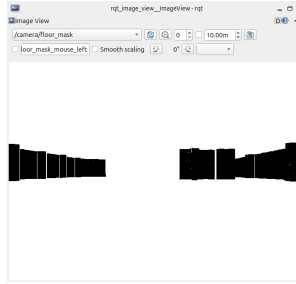


Fig. 2. Inverted Mask

2) *Camera based Error Calculation:* Once a mask has been generated the center of the bottom half of the image is computed to get the path centroid. The formula for centroid computation is,

$$c_x = \frac{M_{10}}{M_{00}} \quad (4)$$

The pixel offset between this centroid and the camera's optical centerline is quantified as the cross-track error e , dictating the desired visual steering angle, where w is the total image width:

$$e = \frac{w_{image}}{2} - c_x \quad (5)$$

Once the error is computed between the image center and the centroid of the path this error is multiplied by a controller value k_p to get the final steering angle/direction.

$$\theta_{cam} = k_p \cdot e_{cam} \quad (6)$$

B. Lidar based navigation

1) *Vector Field Histogram:* Our robot uses lidar sensor to scan a section of front view for detecting obstacles and finding the paths available for navigation. First of all our robot continuously monitors the lidar readings in the operational cone ($\pm \frac{\pi}{4}$ radians).

At every angle within this cone a reading of how far the obstacle is from the robot is recorded within a maximum distance of 4 meter. Once these readings are available they are inverted to create histograms corresponding to each angle.

$$h_i = \frac{1}{r_i + 0.01} \quad (7)$$

Farther objects have smaller histograms while near objects have bigger ones. Then convolution is performed to make the histogram readings smoother. Any histogram above a certain threshold is then masked as blocked while those below are considered free paths as shown below.

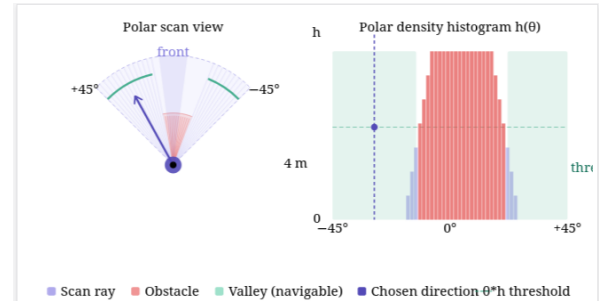


Fig. 3. Vector Field Histogram

Once the valleys are detected the one with the maximum width is chosen and the steering angle is computed by getting the center of the widest valley as shown above.

$$\theta_{lidar} = \frac{w_{widest}}{2} \quad (8)$$

C. SLAM based navigation

In order to overcome the inherent weakness of the VFH algorithm that makes the robot get stuck in local minima SLAM was implemented along with a scoring mechanism to make sure that the robot prefers exploration over already explored regions and obstacles. For any given candidate angle, the algorithm projects a fan of three vectors: one aligned with the trajectory, and two offset laterally (± 0.15 radians) to account for the physical footprint of the robot's chassis. Along each vector, spatial sampling occurs at discrete distance intervals (0.3m, 0.6m, and 0.9m). The system reads the Occupancy Grid cell value at each projected coordinate and applies a heuristic scoring function.

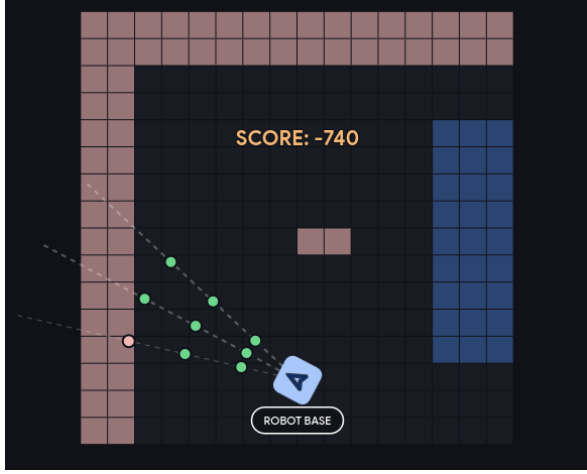


Fig. 4. SLAM negative score



Fig. 5. SLAM positive score

V. CONTROL THROUGH CAMERA AND LIDAR FUSION

The control strategy of the robot depends on the state in which the robot is. We have implemented a state machine comprising the following 6 states in total. These states are shown below:

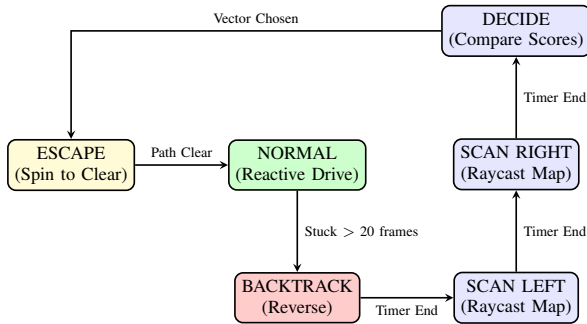


Fig. 6. State transition diagram for the kinematic recovery sequence.

A. Normal State and Dynamic Weighting

In the normal state there are two situations one is when robot is dangerously close (1m) to objects or its

score is less than -500 the final heading of the robot is based entirely on the Lidar angle

$$\theta_{\text{final}} = \theta_{\text{lidar}} \quad (9)$$

when the robot is in the range beyond 2 meters of the obstacle the robot uses a weight of 0.5 for both lidar and camera angles as shown below.

$$\theta_{\text{final}} = 0.5 * \theta_{\text{lidar}} + 0.5 * \theta_{\text{camera}} \quad (10)$$

If the robot is between 1 meter and 2 meter of obstacle dynamic weighting is applied based on the robots distance from the obstacle. In simpler, it performs interpolation between 1 meter and 2 meter to get the weights for lidar and camera.

$$R = \frac{d_{\text{safe}} - d_{\text{min}}}{d_{\text{safe}} - d_{\text{crit}}} \quad (11)$$

$$W_{\text{lidar}} = 0.5 + 0.5R \quad (12)$$

The camera weight is defined as the complement ($W_{\text{cam}} = 1.0 - W_{\text{lidar}}$), yielding the final steering angle θ_{final} :

$$\theta_{\text{final}} = (W_{\text{lidar}} \cdot \theta_{\text{lidar}}) + (W_{\text{cam}} \cdot \theta_{\text{cam}}) \quad (13)$$

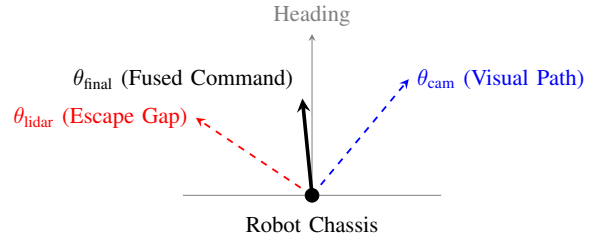


Fig. 7. Dynamic sensor fusion: weighted combination of LiDAR and camera steering angles during an avoidance maneuver.

The final heading is the fusion of camera and lidar based on the robots distance from the obstacle.

B. BACKTRACK STATE:

The robot executes a blind reverse maneuver to clear immediate proximity constraints.

C. SCAN STATE:

The robot systematically pivots in place, executing the raycasting heuristic across adjacent headings and storing the peak map scores.

D. DECIDE STATE:

The system mathematically determines the optimal egress vector by comparing the recorded map scores.

E. ESCAPE STATE:

The system rotates toward the validated heading until spatial constraints dissipate, returning control to the primary reactive logic.

VI. RESULTS

The robot successfully traversed 41 worlds out of 50 starting from 250.

TABLE I
BARN SCORES – SELECTED WORLDS

World	AT (s)	Path (m)	OT (s)	Clipped AT	Score
250	155	32.31	16.16	129.26	0.125
254	66	37.04	18.52	66.00	0.280
256	75	32.73	16.36	75.00	0.218
258	58	51.21	25.61	58.00	0.441
262	72	35.31	17.66	72.00	0.245
264	56	40.38	20.19	56.00	0.361
266	55	48.97	24.49	55.00	0.445
268	84	34.73	17.36	84.00	0.207
270	46	38.90	19.45	46.00	0.423
274	52	36.87	18.44	52.00	0.354
276	72	41.56	20.78	72.00	0.289
280	75	46.46	23.23	75.00	0.310
282	65	40.90	20.45	65.00	0.314
284	54	31.07	15.54	54.00	0.288
286	63	47.80	23.90	63.00	0.379
288	50	43.04	21.52	50.00	0.430
290	62	38.80	19.40	62.00	0.313
292	64	38.63	19.31	64.00	0.302
294	64	38.38	19.19	64.00	0.300
296	70	36.73	18.36	70.00	0.262
Average Score					0.324

VII. CONCLUSION

We built a reactive navigation system for the BARN benchmark that went through eight major versions before settling on a design that works reliably. The core lesson from this project is that reactive systems alone are not enough for highly cluttered, unknown environments—not because they cannot navigate, but because without spatial memory they cannot avoid revisiting the same dead ends repeatedly.

Adding SLAM Toolbox as a map-aware direction scoring layer, rather than as a replacement for the reactive system, proved to be the right call. The VFH gap detection and camera floor following continue to handle moment-to-moment steering, while the occupancy grid provides the higher-level context needed to break out of loops. The result was a 20 percentage point improvement in success rate (54% $\rightarrow$ 74%) with the circling failure mode eliminated entirely.

Limitations

Despite the overall improvement, several limitations remain. The system’s performance degrades in worlds with very narrow corridors where VFH thresholding cannot reliably identify a wide enough free-space valley. The camera masking pipeline is tuned to the specific HSV color distribution of the BARN simulation environment, meaning it may not generalize to real-world deployments without re-tuning. The SLAM scoring heuristic relies on raycasting at fixed angular offsets, which can miss obstacles in geometries where the sampled vectors do not capture the full corridor structure. Additionally, the current state machine uses fixed timers for the backtrack and scan phases, which

may be too short or too long depending on the local environment complexity.

Future Work

The remaining failures—obstacle collisions and dead ends—point toward concrete future directions. Better collision avoidance likely requires either a tighter hard-veto distance threshold or a predictive braking model that accounts for the robot’s deceleration profile. Dead-end recovery could benefit from deliberate path planning within the built SLAM map once sufficient area has been explored, essentially transitioning to a hybrid reactive-deliberate architecture. On the perception side, replacing the fixed HSV masking with a learned segmentation model would improve robustness across different environments and lighting conditions. More principled control strategies such as Model Predictive Control (MPC) or the Dynamic Window Approach augmented with map guidance could also improve performance in the more complex worlds. Finally, adaptive timer durations for the recovery state machine—informed by the local occupancy density—could reduce wasted time during backtrack and scan phases.

REFERENCES

- [1] D. Fox, W. Burgard, and S. Thrun, “The Dynamic Window Approach to Collision Avoidance,” *IEEE Robotics and Automation*, 1997.
- [2] J. Borenstein and Y. Koren, “The vector field histogram-fast obstacle avoidance for mobile robots,” *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.
- [3] M. M. J. Samodro, R. D. Puriyanto, and W. Caesarendra, “Artificial Potential Field Path Planning Algorithm in Differential Drive Mobile Robot Platform for Dynamic Environment,” *International Journal of Robotics and Control Systems*, 2023.
- [4] M. Becker et al., “Obstacle Avoidance Procedure for Mobile Robots,” in *Proc. COBEM*, 2005.
- [5] K. N. McGuire, G. C. H. E. de Croon, and K. Tuyls, “A Comparative Study of Bug Algorithms for Robot Navigation,” *Robotics and Autonomous Systems*, 2018.
- [6] J. H. Rogers III, T. Sellers, T. Lei, C. R. Hudson, and C. Luo, “Centroid-based Cell Decomposition Robot Path Planning Algorithm Integrated with a Bio-Inspired Approach,” in *Proc. SPIE*, 2024.
- [7] Z. Wang, X. Xiao, B. Liu, G. Warnell, and P. Stone, “Benchmarking Metric Ground Navigation,” *arXiv preprint arXiv:2008.13315*, 2020.
- [8] Z. Wang et al., “Autonomous Ground Navigation in Highly Constrained Spaces: Lessons Learned from the 3rd BARN Challenge at ICRA 2024,” *arXiv preprint arXiv:2407.01862*, 2024.
- [9] J. J. Damanik, J.-W. Jung, C. A. Deresa, and H.-L. Choi, “LiCS: Navigation using Learned-imitation on Cluttered Space,” *arXiv preprint arXiv:2406.14947*, 2024.
- [10] ROBOTIS Co., Ltd., “TurtleBot 3 Specifications,” *ROBOTIS e-Manual*, 2024. [Online]. Available: <https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/>